

Módulo: Editor Visual

Paquete: `@mochigo/editor` → carpeta `packages/editor/`

Depende de: prácticamente todos los demás paquetes (`ecs` , `events` , `renderer` , `scenes` , `assets` , `scripting` , `physics` , `animation` , `input` , `sound` , `math`)

Del mapa de arquitectura: nivel más alto — es el único módulo que tiene permiso de depender de todos los demás, porque su función es justamente integrarlos en una interfaz de usuario

1. Responsabilidad exacta

Interfaz visual (dentro del navegador, no una app nativa separada) para: colocar y editar entidades en una escena, inspeccionar y modificar sus componentes (incluyendo scripts del usuario, vía el `schema` declarado en `ScriptComponent` , ver `11-scripting.md`), y guardar/cargar escenas (vía `09-scenes.md`). Es deliberadamente el último módulo en construirse porque depende de que la mayoría de

los demás ya existan y sean estables — no tiene sentido construir un inspector de componentes de Physics si `Collider` / `RigidBody` todavía pueden cambiar de forma.

2. Paneles principales (a construir, cada uno como su propia checklist)

1. **Scene View:** el canvas del juego renderizado en vivo dentro del editor (usa `Renderer` directamente), con capacidad de seleccionar una entidad haciendo click sobre su sprite, y gizmos de transformación (mover/rotar/escalar) sobre la entidad seleccionada.
2. **Hierarchy Panel:** lista de todas las entidades de la escena actual, reflejando la jerarquía padre-hijo de `Transform.parent` (ver `01-ecs.md` sección 6) como un árbol expandible/colapsable.
3. **Inspector Panel:** muestra los componentes de la entidad seleccionada. Para cada componente, genera campos editables automáticamente a partir de su forma de datos — para `ScriptComponent`s del usuario, usa específicamente el `schema` estático declarado (ver `11-scripting.md` sección 2) para saber qué tipo de campo

renderizar (number, string, boolean, vector2, color, entity) y sus límites (min / max).

4. **Asset Browser:** lista los assets cargados/disponibles (vía `AssetManager`), permite arrastrar una textura al Scene View para crear una entidad con `Sprite` apuntando a ella.

3. Interfaz de alto nivel

```
interface EditorConfig {
  world: World;
  eventBus: EventBus;
  renderer: Renderer;
  sceneManager: SceneManager;
  assetManager: AssetManager;
  mountElement: HTMLElement;
}

class Editor {
  constructor(config: EditorConfig);

  mount(): void;    // renderiza la UI del
editor dentro de mountElement
  unmount(): void;

  selectEntity(entity: EntityId | null):
void;
  getSelectedEntity(): EntityId | null;
```

```
// Entra/sale de modo "play" – al entrar,
// el editor deja que el Game Loop corra
// normalmente (los scripts se ejecutan);
// al salir, vuelve al estado exacto de
// antes de entrar en modo play (requiere
// snapshot del World, ver sección 4)
enterPlayMode(): void;
exitPlayMode(): void;
isInPlayMode(): boolean;
}
```

4. Decisión de arquitectura: play mode con snapshot/restore

Patrón estándar en editores de motores de juego (Unity, Godot): el desarrollador debe poder darle "play" a la escena que está editando para probarla, y al salir de play, volver exactamente al estado en que estaba antes de probar (no que los cambios que hicieron los scripts durante la prueba queden permanentes). Para esto:

- Al llamar `enterPlayMode()`, el editor usa `SceneManager.serializeCurrentScene()` (ver `09-scenes.md` sección 3) para tomar una "foto" del estado actual del `World`, y la guarda en memoria (no en `Storage`, es temporal).

- Al llamar `exitPlayMode()`, el editor llama `SceneManager.LoadScene()` con esa foto guardada, restaurando el `World` exactamente a como estaba.

Esto reutiliza directamente la infraestructura de serialización de escenas — deliberado, para no duplicar lógica de snapshot en el Editor.

5. Eventos

El Editor mayormente **escucha** eventos de todos los demás módulos para mantener su UI sincronizada (`ecs:entity-created`, `ecs:component-added`, `scene:loaded`, etc. — ver tabla central en `03-event-manager.md`) más que emitir eventos propios nuevos. Si durante la implementación surge la necesidad de un evento propio del editor (por ejemplo, `editor:selection-changed`), debe añadirse a la tabla central siguiendo el proceso descrito en `03-event-manager.md` sección 3.

6. Checklist de implementación

- [] Clase `Editor` con la interfaz de la sección 3

- [] **Scene View**: renderizado en vivo del `World` actual dentro del editor, selección de entidad por click, gizmos de mover/rotar/escalar sobre la entidad seleccionada que escriben directamente sobre su `Transform`
- [] **Hierarchy Panel**: árbol de entidades reflejando `Transform.parent`, actualizado en vivo al escuchar `ecs:entity-created` / `ecs:entity-destroyed`
- [] **Inspector Panel**: generación automática de campos editables a partir de:
 - [] componentes "core" definidos por otros módulos (Transform, Sprite, Camera, Rigidbody, Collider, Animator) – cada uno con su propio formulario apropiado a su forma de datos
 - [] `ScriptComponent`s del usuario, leyendo su `schema` estático según `11-scripting.md` sección 2, generando el tipo de campo correcto por cada `SchemaFieldType` (number con min/max, string, boolean, vector2 como par de inputs numéricos, color como color picker, entity como selector de entidad)
- [] **Asset Browser**: lista de assets cargados vía `AssetManager`, con

drag-and-drop hacia el Scene View para crear una entidad `Sprite` nueva

- [] `enterPlayMode()` / `exitPlayMode()` con snapshot/restore según sección 4
- [] Botón/acción de "Guardar escena" que llama `SceneManager.serializeCurrentScene()` y lo persiste (a través del módulo `Storage` o descargándolo como archivo JSON — decidir la UX exacta durante la implementación, documentar la decisión)
- [] Tests: seleccionar una entidad en la Hierarchy la selecciona también en el Inspector y en el Scene View (estado de selección compartido y consistente)
- [] Tests: modificar un campo en el Inspector efectivamente actualiza el componente correspondiente en el `World` (no solo visualmente en la UI)
- [] Tests: `enterPlayMode()` seguido de cambios (simulando que un script movió una entidad) y luego `exitPlayMode()` restaura la posición original exacta
- [] Tests: el Inspector genera el tipo de campo correcto para cada `SchemaFieldType` posible, usando un `ScriptComponent` de prueba con un schema que cubra los 6 tipos

